MACHINE LEARNING PIPELINE

Dataset: Metro Interstate Traffic Volume — hourly westbound I-94 traffic, weather and holiday data from Minneapolis–St. Paul, Minnesota (UCI Machine Learning Repository) dataset

Prediction target: Hourly traffic volume (`traffic_volume`), measured in vehicles per hour

Repository: GitHub

Group members: [Emmanuel Oluwajomiloju Akinloye, Larissa Inyange, Jonathan Mugisha, Patrick Tuyizere]

Date: 30 October 2026

Summary

This report presents the results of a full time-series pipeline designed from hourly traffic counts from Interstate 94 near Minneapolis–St Paul spanning six years, and with local meteorological data. It includes the four steps of the assignment: Data preparation, to a script that predicts the next hour of traffic using live API data.

Task	What was built	Headline result
1. Preprocessing, analysis, model	Cleaned the series to a complete hourly index, answered 5 analytical questions (with lags and moving averages), trained and compared 2 Random Forest experiments	Adding lagged traffic values and a rolling average calculated only from earlier hours reduced MAE from 438.77 to 156.62 vehicles per hour, while R^2 increased from 0.8347 to 0.9851.
2. Databases	MySQL with 4 normalised tables and an ERD; MongoDB collection with embedded weather. Each has 4	Both databases hold the same 40,575 hourly records and return identical

	queries and full CRUD examples	answers to the same questions
3. API	FastAPI service with POST, GET, PUT, DELETE, latest-record and date-range endpoints for both databases	Same JSON contract for MySQL and MongoDB, so clients can switch source with one parameter
4. Prediction script	Fetches recent records from the API, applies the Task 1 preprocessing, loads the saved model and forecasts 1 to 48 hours ahead	The operational benchmark achieved a one-hour-ahead MAE of 171.87 vehicles per hour. Recursive forecast error increased to 289.66 vehicles per hour at the 24-hour horizon.

Two limitations the reader should know about

(1) The outage of the sensor from August 2014 to June 2015 (307 days) is filled with the constant value, so there are about 26% of artificial training rows. The reported accuracy for the test period (2018) was valid as it is based on 99.7% real data, although a model trained on real data might perform better. The model utilizes the weather of the hour that it is trying to predict, and this is not known at the time of the forecast, thus the prediction script assumes the last observed weather. Both are discussed in Section 7.

1. Dataset and objective

The Metro Interstate Traffic Volume dataset records the number of vehicles passing a sensor on I-94 westbound each hour, with the weather reported for that hour. The objective is to forecast traffic_volume for the next hour using past traffic, weather and calendar information. The timestamp column is date_time.

1.1 Columns

Column	Type	Meaning
date_time	Timestamp	Local hour of the observation
traffic_volume	Integer	Vehicles in that hour (target), 0 to 7,280

temp	Float	Temperature in Kelvin
rain_1h, snow_1h	Float	Rain and snow in the hour, in mm
clouds_all	Integer	Cloud cover, 0 to 100%
weather_main, weather_description	Text	Weather category and detailed description
holiday	Text	Name of a US holiday, otherwise None

2. Task 1: Preprocessing and exploratory analysis

2.1 Understanding the dataset

Question	**Finding**
Time range	2 October 2012, 09:00 to 30 September 2018, 23:00 (about 6 years)
Frequency	Hourly. Timestamps are not continuous: sensor outages leave gaps
Raw rows	48,204
Unique timestamps	40,575 unique timestamps. The remaining 7,629 rows are additional weather reports associated with timestamps that already occur in the dataset.
Expected hourly slots	52,551, so 11,976 hours (22.8%) have no observation
Largest gap	307 days and 19 hours, from 8 August 2014 at 01:00 to 11 June 2015 at 20:00, containing 7,386 missing hourly slots.
Typical gaps	Most timestamp jumps are short: 2,192 jumps of two hours, 201 jumps of three hours and 59 jumps of four hours.
Target	Mean 2,901 vehicles/hour, median 2,883, standard deviation 2,083, range 0 to 7,280

Data quality issues and how each was handled

Issue	**Evidence**	**Treatment and reason**

Holiday read as missing	48,143 None strings were parsed as NaN	Kept as the category None and added an is_holiday flag. These are not missing values, they mean "not a holiday"
Duplicate timestamps	7,629 additional rows occur at timestamps that already contain another weather report, because one hour can have several weather descriptions.	Averaged the numeric columns per hour. A time series needs exactly one value per hour
Missing hours	11,976 empty hourly slots	Re-indexed to a complete hourly grid. Temperature and clouds: linear interpolation over time, because weather changes smoothly. Rain and snow: 0. Traffic: forward-fill then back-fill
Sensor faults	10 rows with temp = 0 K; one rain reading of 9,831 mm	Flagged as NaN before imputation (temp < 100 K, rain > 500 mm). These values are physically impossible

Weather variables are heavily skewed towards zero: snow_1h is 0 in almost every hour and the median rain_1h is 0. Cloud cover is spread across the full 0 to 100 range, and temperature is roughly bell-shaped with a mean near 282 K (9 °C) and a strong seasonal swing.

2.2 Analytical questions

Five questions were answered, each with a visualization and interpretation. Questions 1 and 2 use a moving average and lagged features as required. All figures use the cleaned hourly series (52,551 hours).

Question 1. Is there an increasing or decreasing trend, and does a moving average make it easier to see?

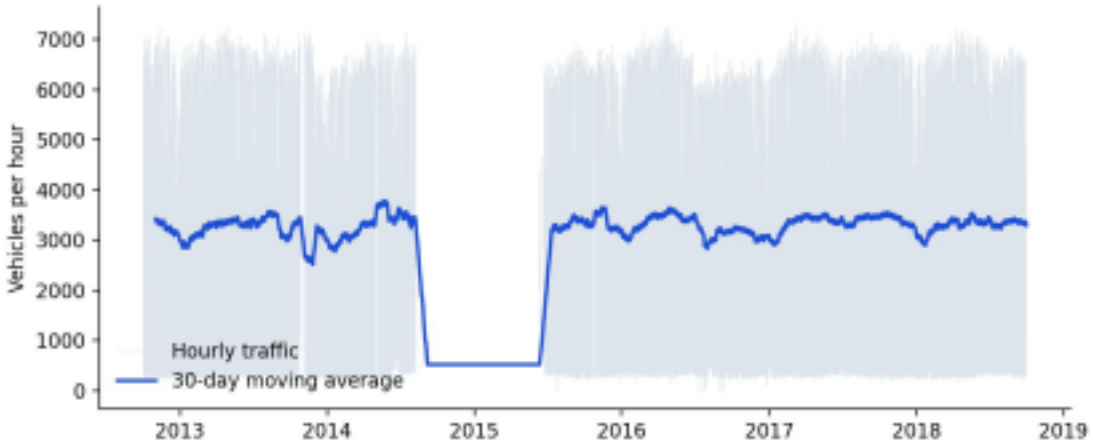


Figure 1. Hourly traffic with a 30-day (720-hour) moving average, 2012 to 2018

Hourly traffic swings between near zero at night and about 7,000 at rush hour, which hides any long term direction. The 30-day moving average removes that daily cycle and shows a **stable, flat level of roughly 3,000 to 3,500 vehicles per hour** with no sustained upward or downward trend. Dips appear around the turn of each year, which is consistent with winter conditions and holidays. The collapse
between August 2014 and June 2015 is **not a real traffic drop**: it is the 307-day sensor outage, which was filled with a constant value during preprocessing.

Question 2. Is traffic now related to traffic in previous hours (lag effects)?

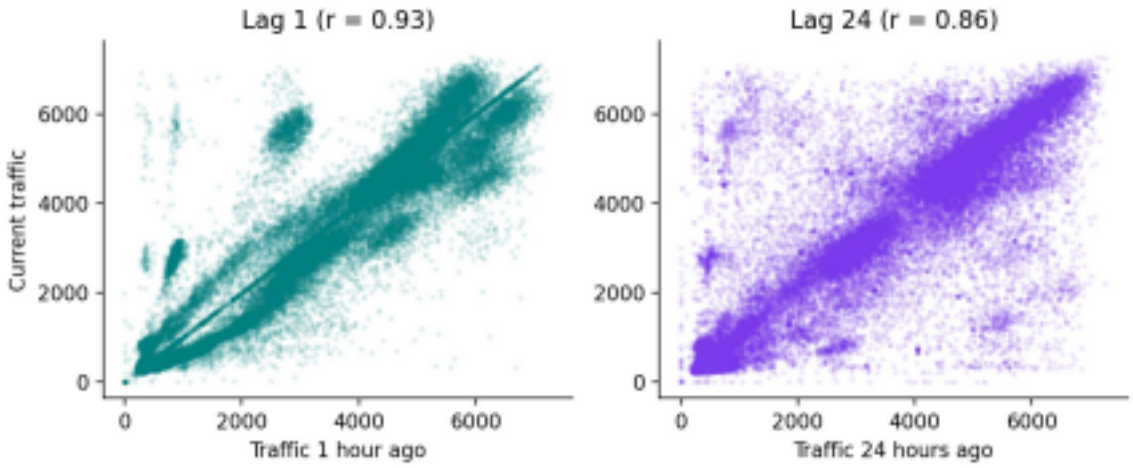


Figure 2. Current traffic against traffic 1 hour ago and 24 hours ago

The traffic is highly auto-correlated. The correlation is 0.93 with the previous hour, 0.86 with the same hour yesterday and 0.90 with the same hour last week. The lag 1 scatter graph reveals two high density clusters: the first between 0 and 1000 vehicles at night; the second between 4500 and 6000 vehicles during the day. Traffic does not magically transform itself from one hour to the next, so lag 1 is a great predictor. The weekly lag is important because it is more pronounced than the daily lag, which accounts for the fact that there are different patterns on weekdays and weekends. This motivated using lags of 1, 24 and 168 hours as model features.

Question 3. Do weather variables correlate with traffic over time?

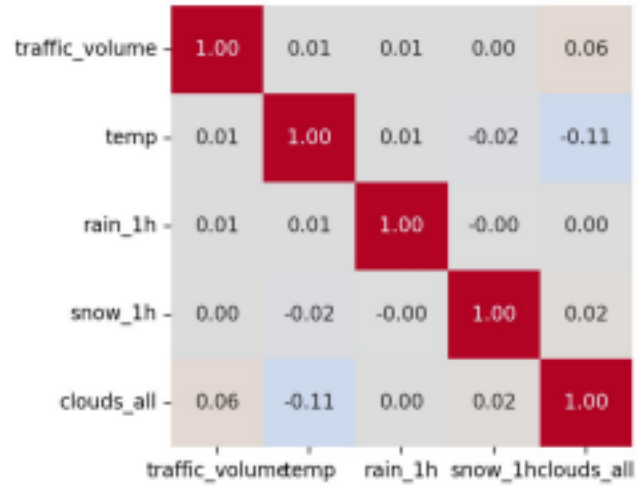


Figure 3. Correlation between traffic volume and weather variables

The correlation of traffic with temperature, rain, snow and cloud cover is **close to zero**. Weather does not move traffic in a simple linear way at the hourly level, because the daily commuting cycle dominates: traffic is high at 5 pm in good weather and in bad weather alike. A weak seasonal effect exists, as seen in the winter dips of Figure 1, but the hour-to-hour noise masks it. Weather therefore adds little on its own, which is later confirmed by the model's feature importances.

Question 4. Which hours of the day are busiest and quietest?

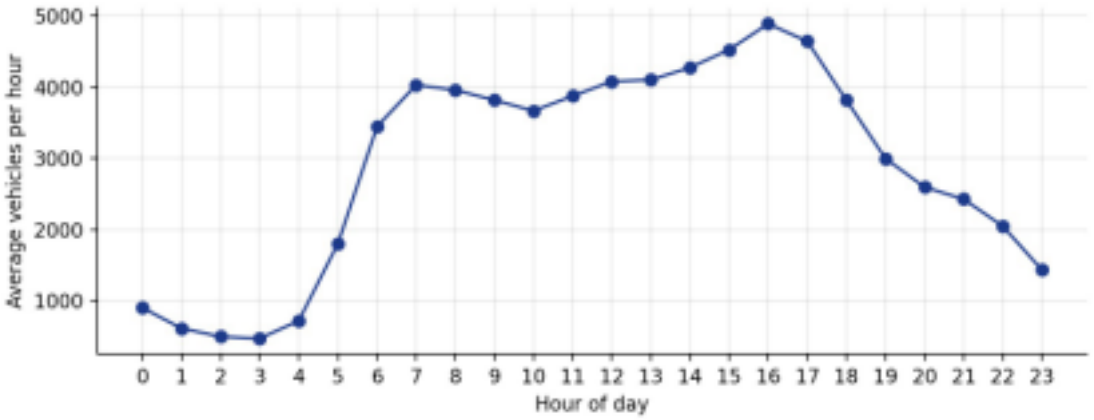


Figure 4. Average traffic volume by hour of the day

Traffic follows a commuter pattern. The quietest hours are **2 to 3 am (about 470 to 490 vehicles)**. Volume rises sharply from 5 am to a morning peak of about **4,030 at 7 am**, stays high through midday, and reaches the daily maximum of about **4,890 at 4 pm (hour 16)** during the evening commute. It then falls steadily towards midnight.

Question 5. How does traffic differ between weekdays and weekends?

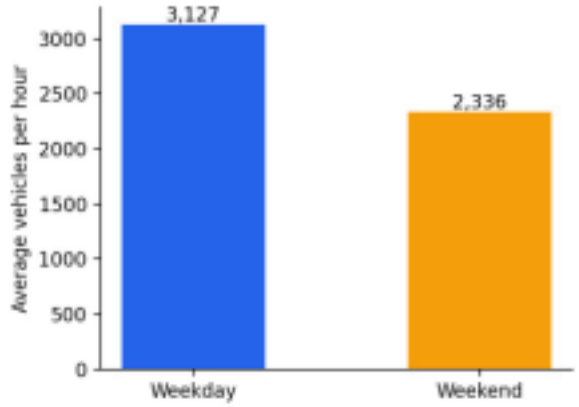


Figure 5. Average hourly traffic on weekdays and weekends

Weekdays average about **3,127 vehicles per hour** against **2,336 at weekends**, roughly 25% less. Weekday traffic is driven by commuting to work and school, while weekend travel is more spread out. This is why the model receives dayofweek and is_weekend as inputs.

A note on figures across sections

Averages in this section include forward-filled hours, which are mostly the flat 2014 to 2015 outage period and pull averages down. The SQL queries in Section 3 average only observed hours, so the busiest hour reads 5,709 vehicles there against 4,891 here. Both are correct for the data they use.

2.3 Training a model

A Random Forest regressor was chosen because it captures non-linear interactions between hour, day and traffic history and needs no feature scaling. The data was split **chronologically**: everything before 1 January 2018 for training (45,831 hours) and 2018 onwards for testing (6,552 hours). A random split would leak future information into training.

Features

- **Calendar:** hour, day of week, weekend flag
- **Weather:** temperature, cloud cover, rain
- **Lag features:** traffic 1 hour, 24 hours and 168 hours earlier
- **Moving average:** 24-hour rolling mean of traffic

Experiments

	Experiment 1: baseline	**Experiment 2: tuned, with lags**
Features	hour, dayofweek, temp, clouds_all, rain_1h (5)	The 5 above, plus is_weekend, lag_1, lag_24, lag_168, rolling_mean_24 (10)
n_estimators	50	100
max_depth	8	18
min_samples_leaf	1 (default)	4
MAE (vehicles/hour)	438.77	156.62
RMSE (vehicles/hour)	803.68	241.66
R^2	0.8347	0.9851

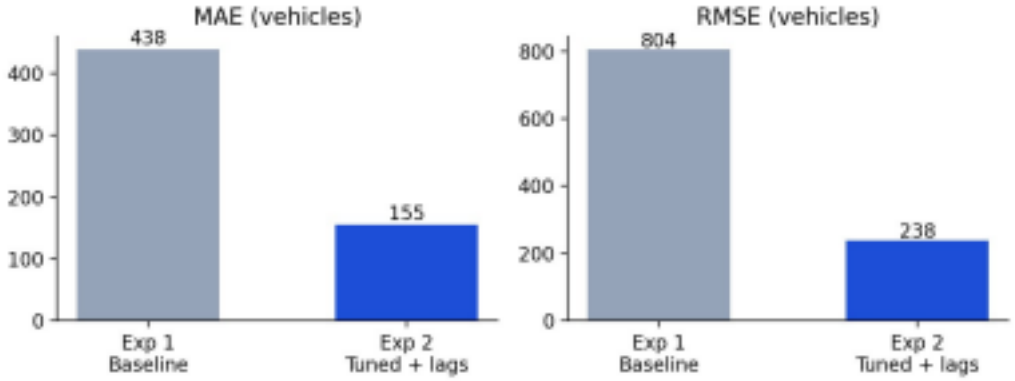


Figure 6. Error of the two experiments on the 2018 test set

Experiment 2 reduced MAE by approximately 64.3%, from 438.77 to 156.62 vehicles per hour. RMSE decreased by approximately 69.9%, from 803.68 to 241.66, while R^2 increased from 0.8347 to 0.9851. The rolling average excludes the target hour, preventing target leakage.

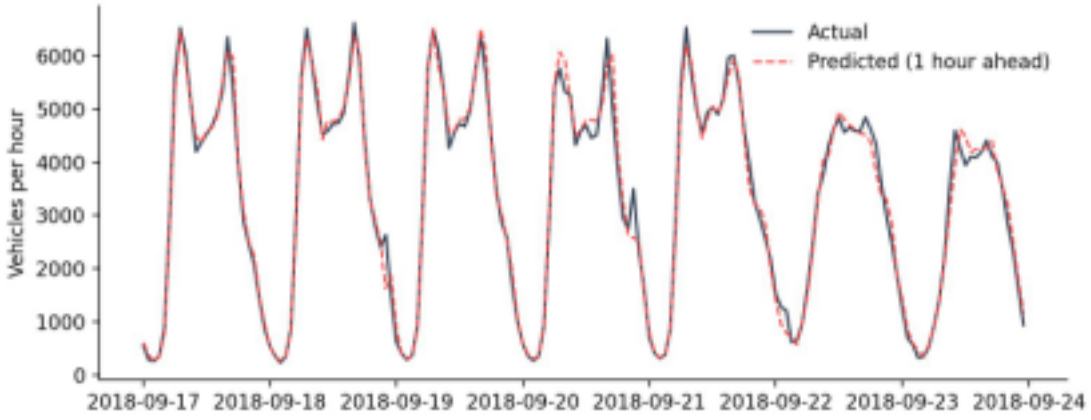


Figure 7. One test week (17 to 23 September 2018): actual traffic against one-hour-ahead predictions

The predicted line follows both the weekday double peaks and the flatter weekend days closely. The largest errors occur at the sharp morning and evening ramps, where traffic changes fastest.

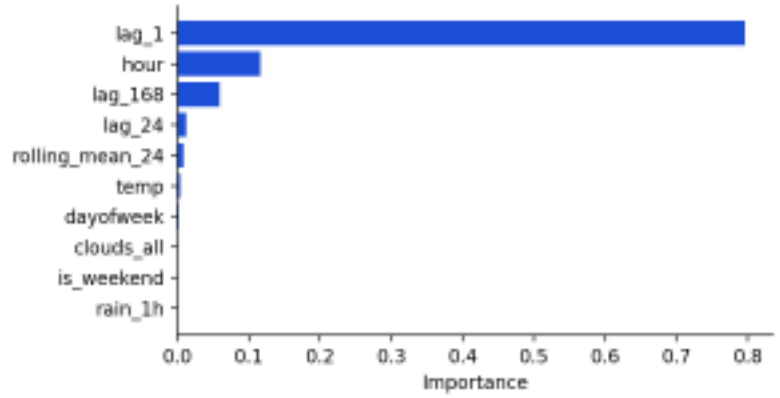


Figure 8. Feature importance of the saved model

lag_1 carries about 80% of the importance, followed by hour (12%) and lag_168 (6%). Weather variables together contribute under 1%, in line with the weak correlations in Question 3. Experiment 2 was saved as traffic_forecasting_model.pkl.

3. Task 2: Database design

These source observations were cleaned and restructured for the respective databases, and then loaded into MySQL and MongoDB. The database pipeline did not produce a full hourly grid or fill in missing hours as was done in Task 1. MySQL has 40,575 traffic observations and 48,187 weather readings; MongoDB has 40,575 documents containing arrays of weather readings.

3.1 Relational schema (MySQL)

The schema has four tables in third normal form. holidays and weather_conditions are lookup tables so each name is stored once. traffic_observations has one row per hour. weather_readings has one row per weather report, because one hour can have several (for example rain and mist together).

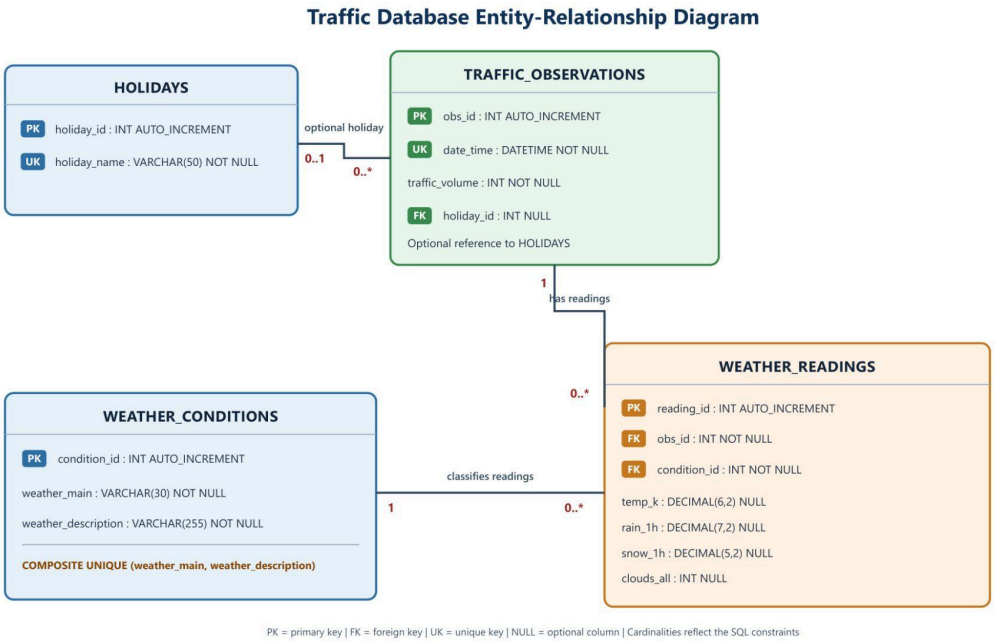


Figure 9. Entity-relationship diagram

Table	Rows	Key columns
holidays	11	holiday_id (PK), holiday_name (unique)
weather_conditions	37	condition_id (PK), weather_main, weather_description (unique together)
traffic_observations	40,575	obs_id (PK), date_time (unique), traffic_volume, holiday_id (FK, nullable)
weather_readings	48,187	reading_id (PK), obs_id (FK), condition_id (FK), temp_k, rain_1h, snow_1h, clouds_all

Relationships: one holiday to many observations; one observation to many weather readings; one weather condition to many readings. The unique constraint on date_time prevents two traffic observations from sharing the same timestamp and speeds up date-range queries. It does not guarantee continuous hourly coverage because some

timestamps are missing from the source data.

```
CREATE TABLE traffic_observations (
 obs_id INT AUTO_INCREMENT PRIMARY KEY,
 date_time DATETIME NOT NULL UNIQUE,
 traffic_volume INT NOT NULL,
 holiday_id INT,
 FOREIGN KEY (holiday_id) REFERENCES holidays(holiday_id)
 );

CREATE TABLE weather_readings (
 reading_id INT AUTO_INCREMENT PRIMARY KEY,
 obs_id INT NOT NULL,
 condition_id INT NOT NULL,
 temp_k DECIMAL(6,2), rain_1h DECIMAL(7,2),
 snow_1h DECIMAL(5,2), clouds_all INT,
 FOREIGN KEY (obs_id) REFERENCES traffic_observations(obs_id),
 FOREIGN KEY (condition_id) REFERENCES weather_conditions(condition_id) );
```

The full script, including the two lookup tables, is in database/sql/schema.sql.

3.2 MySQL queries and results

Query 1. Average traffic by weather type (joins three tables)

```
SELECT wc.weather_main, ROUND(AVG(t.traffic_volume)) AS avg_traffic, COUNT(*) AS  n_readings
FROM traffic_observations t
JOIN weather_readings r ON r.obs_id = t.obs_id
JOIN weather_conditions wc ON wc.condition_id = r.condition_id
GROUP BY wc.weather_main ORDER BY avg_traffic DESC;
```

weather_main	avg_traffic	n_readings
clouds	3,618	15,158
haze	3,502	1,360
rain	3,318	5,672
drizzle	3,292	1,820
smoke	3,238	20
clear	3,056	13,384

snow	3,016	2,875
thunderstorm	2,999	1,033
mist	2,933	5,949
fog	2,704	912
squall	2,062	4

Fog, squalls and mist coincide with the lowest traffic, while cloudy and hazy hours are the busiest. This is consistent with the weak weather signal in Section 2: differences between weather types are modest, and fog and squalls are rare. An hour with two readings counts once per reading.

Query 2. Holiday traffic compared with normal days

Holidays are only flagged on the midnight hour, so both sides are limited to hour 0 for a fair comparison.

```
SELECT COALESCE(h.holiday_name, 'Not a holiday') AS day_type, COUNT(*) AS n_hours, ROUND(AVG(t.traffic_volume)) AS avg_midnight_traffic
FROM traffic_observations t LEFT JOIN holidays h ON h.holiday_id = t.holiday_id WHERE HOUR(t.date_time) = 0 GROUP BY day_type ORDER BY avg_midnight_traffic DESC;
```

day_type	n_hours	avg_midnight_traffic
New Years Day	5	1,325
Memorial Day	5	1,108
Independence Day	5	1,076
Labor Day	5	1,013
Thanksgiving Day	5	954
Christmas Day	5	847
Not a holiday	1,670	844
Veterans Day	5	691
Washingtons Birthday	5	635

State Fair	5	635
Martin Luther King Jr Day	3	593
Columbus Day	5	519

Midnight traffic is higher after celebration holidays (New Year, Memorial Day, Independence Day, Labor Day) and lower on quiet single-day holidays such as Columbus Day. Each holiday has only 3 to 5 samples, so these are indicative rather than conclusive.

Query 3. The five busiest hours of the day

```
SELECT HOUR(date_time) AS hour_of_day, ROUND(AVG(traffic_volume)) AS avg_traffic FROM traffic_observations GROUP BY hour_of_day ORDER BY avg_traffic DESC LIMIT 5;
```

hour_of_day	avg_traffic
16	5,709
17	5,351
15	5,272
14	4,957
7	4,770

The afternoon commute (14:00 to 17:00) dominates, with the morning peak at 7 am in fifth place.

Query 4. Data-quality check for NULL sensor values

```
SELECT SUM(temp_k IS NULL) AS null_temp, SUM(rain_1h IS NULL) AS null_rain FROM weather_readings;
```

null_temp	null_rain
10	1

These match the 10 impossible temperatures and the single rain spike found in Task 1, confirming the faulty values were stored as NULL.

Create, update and delete

A test observation for 1 October 2018, 00:00 (after the last real row) was created, corrected and removed. Because of the foreign key, the weather reading must be deleted before its parent

observation.

```
INSERT INTO traffic_observations (date_time, traffic_volume, holiday_id) VALUES ('2018-10-01 00:00:00', 450, NULL);
INSERT INTO weather_readings (obs_id, condition_id, temp_k, rain_1h, snow_1h, clouds_all)
VALUES (<new obs_id>, <id of 'sky is clear'>, 285.50, 0, 0, 20); -- 1 row affected each
UPDATE traffic_observations SET traffic_volume = 475 WHERE date_time = '2018-10-01 00:00:00'; -- 1 row affected
DELETE FROM weather_readings WHERE obs_id = (SELECT obs_id FROM traffic_observations WHERE date_time = '2018-10-01 00:00:00');
DELETE FROM traffic_observations WHERE date_time = '2018-10-01 00:00:00'; -- Afterwards: 0 rows for that hour; totals back to 40,575 observations and 48,187 readings
```

3.3 MongoDB collection design

MongoDB uses a single collection, traffic_observations, with **one document per hour and the weather readings embedded as an array**. Weather readings belong to exactly one hour and are always read together with it, so embedding avoids the joins the SQL design needs. A unique index on date_time provides fast range queries.

```
{
 "date_time": ISODate("2012-10-10T07:00:00Z"),
 "traffic_volume": 6793,
 "weather": [
 { "main": "rain", "description": "light rain", "temp_k": 281.25, "rain_1h": 0.0, "snow_1h": 0.0, "clouds_all": 99 },
  { "main": "drizzle", "description": "light intensity drizzle", "temp_k": 281.25, "rain_1h": 0.0, "snow_1h": 0.0, "clouds_all": 99 }
 ]
}
```

The holiday field is only present on holiday hours, and a sensor error is represented by leaving the field out rather than storing a false value. A $jsonSchema validator requires date_time, traffic_volume (integer, at least 0) and a non-empty weather array whose items need main, description and clouds_all, and it limits clouds_all to 0 to 100.

Query 1. Five busiest hours ever (find with filter, projection, sort, limit)

```
db.traffic_observations.find({traffic_volume: {$gt: 7000}}, {_id: 0, date_time: 1, traffic_volume: 1})
 .sort({traffic_volume: -1}).limit(5)
```

date_time	traffic_volume

2017-03-09 16:00	7,280
2016-04-21 07:00	7,260
2016-04-22 07:00	7,241
2013-04-30 07:00	7,217
2018-04-12 16:00	7,213

The busiest hours are all weekday rush hours, at 7 am or 4 pm, and all fall in spring.

Query 2. Average traffic by weather type (aggregation with $unwind)

```
db.traffic_observations.aggregate([
 {$unwind: '$weather'},
 {$group: {_id: '$weather.main', avg_traffic: {$avg: '$traffic_volume'}, n_readings:  {$sum: 1}}},
 {$sort: {avg_traffic: -1}} ])
```

Returns the same 11 weather types and values as MySQL Query 1 (clouds 3,618 down to squall 2,062), which confirms the two databases agree.

Query 3. Average traffic by hour of the day (aggregation with $hour)

```
db.traffic_observations.aggregate([
 {$group: {_id: {$hour: '$date_time'}, avg_traffic: {$avg: '$traffic_volume'}}},  {$sort: {avg_traffic: -1}}, {$limit: 5}
])
```

Returns hours 16, 17, 15, 14 and 7 with averages 5,709, 5,351, 5,272, 4,957 and 4,770, identical to MySQL Query 3.

Query 4. Holiday traffic compared with normal days

```
db.traffic_observations.aggregate([
 {$match: {$expr: {$eq: [{$hour: '$date_time'}, 0]}}},
  {$group: {_id: {$ifNull: ['$holiday', 'Not a holiday']}, n_hours: {$sum: 1},  avg_midnight_traffic: {$avg:
'$traffic_volume'}}},
 {$sort: {avg_midnight_traffic: -1}} ])
```

Returns the same 12 groups as MySQL Query 2, from New Years Day (1,325) to Columbus Day (519), with "Not a holiday" at 844 across 1,670 hours.

Create, update and delete

```
db.traffic_observations.insertOne({date_time: ISODate('2018-10-01T00:00:00Z'), traffic_volume: 450,
 weather: [{main: 'clear', description: 'sky is clear', temp_k: 285.5, rain_1h: 0, snow_1h: 0, clouds_all: 20}]})
db.traffic_observations.updateOne({date_time: ISODate('2018-10-01T00:00:00Z')}, {$set: {traffic_volume: 475}})
db.traffic_observations.updateOne({date_time: ISODate('2018-10-01T00:00:00Z')}, {$push: {weather: {main:
'mist', ...}}})
db.traffic_observations.deleteOne({date_time: ISODate('2018-10-01T00:00:00Z')}) // one command removes hour
and weather
```

3.4 MySQL compared with MongoDB

Aspect	**MySQL**	**MongoDB**
Structure	4 normalised tables, joined at query time	1 collection, weather embedded in the hour
Reading one hour with its weather	Join across 3 tables	One document read
Deleting an hour	Two deletes (child rows first) because of the foreign key	One command
Data integrity	Enforced by keys and constraints	Enforced by the $jsonSchema validator
Aggregations across hours	Standard SQL, very readable	Pipelines with $unwind and $group, flexible for nested data
Best fit here	Reporting and consistency across lookups	Serving whole hourly records to an API or model

The data-loading notebook had an in-memory MongoDB emulator to which database validators weren't applied. The repository contains therefore the following file, named "database/mongodb/setup_collection.js": this file can be used with a real MongoDB server. This script will create/update the strict $jsonSchema validator, and generate the unique date_time index. With the validator installed, the restored collection was checked to have 40,575 documents.

4. Task 3: API for CRUD and time-series queries

A FastAPI application exposes both databases through the same set of endpoints. The two route groups share one data contract, so a client can switch databases by changing mysql to mongodb in the URL.

4.1 Endpoints

Method	Path (prefix /api/mysql/records or /api/mongodb/records)	Purpose	Status
POST	/	Create a record (hour plus its weather readings)	201
GET	/latest	Latest record by timestamp	200
GET	/range?start=...&end=...	Records between two timestamps, in time order	200

GET	/{id}	One record by id	200
PUT	/{id}	Replace a whole record	200
DELETE	/{id}	Delete a record	200
GET	/ and /health	Service information and database connectivity check	200

4.2 Validation and errors

- Request bodies are validated with Pydantic: traffic_volume at least 0, clouds_all between 0 and 100, rain and snow not negative, and at least one weather reading per record. • Weather text is trimmed and lower-cased. Timezone-aware timestamps are converted to UTC so both databases store the same naive timestamp.
- A duplicate timestamp on POST returns **409 Conflict**. A range with start after end returns **400**. Missing records return **404**. Database failures return **503**.
- MySQL ids are integers and MongoDB ids are ObjectId strings, so the response id field accepts both.

4.3 Example response

GET /api/mongodb/records/latest returns the last hour in the dataset:


```
{
 "id": "6ae45d075b72d1ec79484ee3",
 "date_time": "2018-09-30T23:00:00",
 "traffic_volume": 954,
 "holiday": null,
 "weather": [ { "main": "clouds", "description": "overcast clouds", "temp_k": 282.12, "rain_1h": 0.0, "snow_1h": 0.0, "clouds_all": 90 } ] }
```


GET /api/mysql/records/latest returns the same record and the same shape, with an integer id. The repository also contains automated tests for the request schemas and for the endpoints (api/tests). Interactive documentation is generated at /docs when the server runs.

5. Task 4: Prediction and forecast script

task4_prediction/predict_traffic.py ties the earlier tasks together. One run performs the four

required steps:

Step	What the script does
1. Fetch	Calls /latest to find the most recent hour, then /range for the previous 8 days (192 hours) from MySQL or MongoDB
2. Preprocess	Applies the Task 1 rules: averages the weather readings inside each hour, removes sensor faults (temp < 100 K, rain > 500 mm), builds a complete hourly index, interpolates temperature and clouds, sets rain and snow to 0, forward-fills traffic, then computes hour, day of week, weekend flag, lags 1, 24 and 168, and the 24-hour moving average
3. Load model	Loads traffic_forecasting_model.pkl with joblib and reads the expected feature names from the model itself
4. Predict	Predicts the next hour, or up to 48 hours using a recursive forecast in which each prediction feeds the next step's lags

5.1 Running it

```
python task4_prediction/predict_traffic.py # next hour, MySQL python task4_prediction/predict_traffic.py --source mongodb --horizon 6 python task4_prediction/predict_traffic.py --as-of 2018-09-20T06:00:00 --horizon 6 # backtest
```

Default run against the last hour in the database:

```
Traffic forecast | source: mongodb | model: traffic_forecasting_model.pkl Last observed hour : 2018-09-30 23:00 (954 vehicles)
History used : 193 records, 2018-09-22 23:00 -> 2018-09-30 23:00, 0 missing hour(s) filled
Weather assumed : temp 282.12 K, rain 0.0 mm, clouds 90.0%

Hour Predicted
2018-10-01 00:00 501
```

Because the data ends at 30 September 2018, there is no real value to compare with for that forecast. The --as-of option runs the same pipeline as if an earlier hour were the latest, and shows the real values next to the forecast:

Hour (Thursday 20 Sep 2018)	Predicted	Actual	Error
07:00	6,106	5,766	+340
08:00	5,903	5,331	+572
09:00	5,411	5,264	+147
10:00	4,557	4,326	+231
11:00	4,768	4,592	+176
12:00	4,885	4,695	+190

The forecast follows the morning rush shape (peak, then a decline towards midday) but over-predicts every hour, by 276 vehicles on average. This is one sample, not an accuracy measure; the next section measures accuracy properly.

5.2 How accurate is the forecast, and how does error grow with the horizon?

The recursive forecasting pipeline was evaluated using 207 deterministic starting hours selected evenly across 6,223 eligible hourly origins in 2018. An origin was eligible only when observed traffic values existed for each of the following 24 hours. For every origin, the pipeline used only information available at that time and carried the last observed weather forward during the forecast. Mean absolute error was calculated separately at the exact 1-, 3-, 6-, 12-, and 24-hour forecast leads. The complete predictions and errors are stored in forecast_benchmark_details.csv, while the aggregated results are stored in forecast_benchmark_summary.csv.

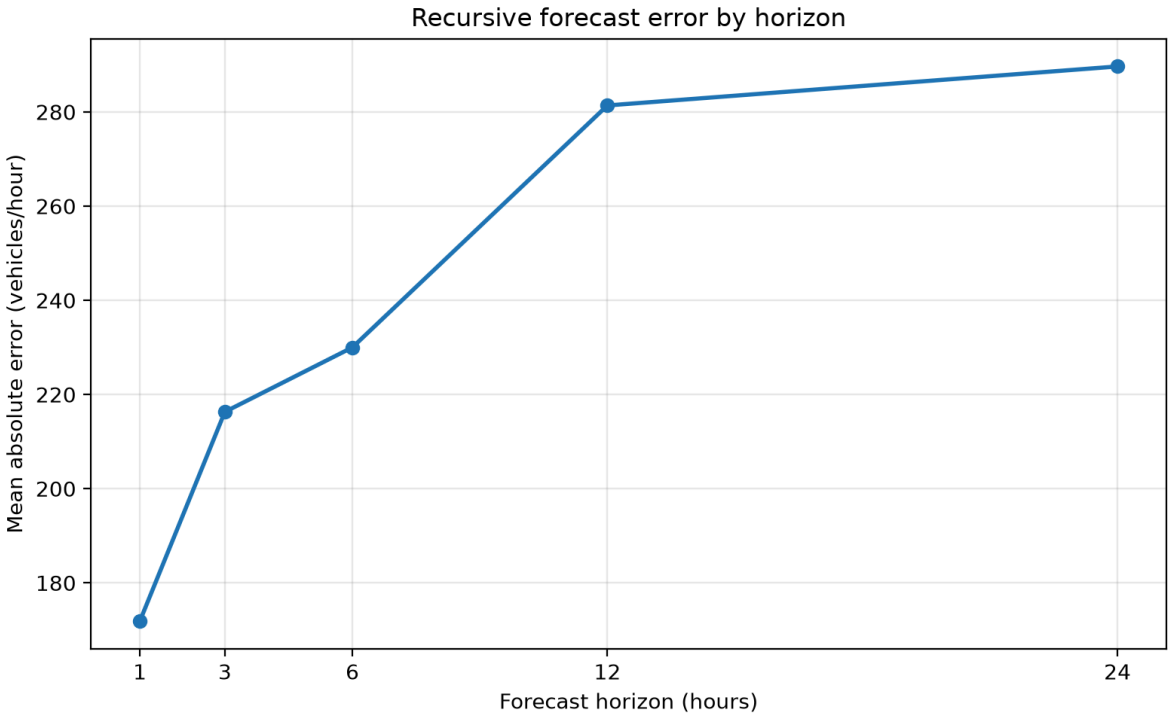


Figure 10. Exact-lead mean absolute error of the recursive forecast at five horizons, evaluated using 207 deterministic starting points in 2018.

Horizon	**MAE (vehicles/hour)**
1 hour	171.87
3 hours	216.31
6 hours	230.00
12 hours	281.38
24 hours	289.66

The one-hour-ahead forecast achieved the lowest MAE at 171.87 vehicles per hour. Error generally increased with the forecast horizon, reaching 289.66 vehicles per hour at 24 hours. This occurs because the forecasting process is recursive: predictions from earlier steps are used to construct features for later steps, allowing errors to propagate. The results were calculated from 207 deterministic starting points in 2018 and can be reproduced by running task4_prediction/benchmark_forecasts.py.

5.3 Design decisions

- **Same preprocessing as Task 1.** A test compares the features built by the script from the database records with the Task 1 processed file for the same hour: lags and calendar features match exactly.
- **Moving average over earlier hours only.** Both Task 1 and Task 4 calculate rolling_mean_24 from the 24 hours before the target timestamp. The target hour is excluded, preventing target leakage. After retraining with this correction, the Task 1 model achieved a one-hour holdout MAE of 156.62 vehicles per hour. The Task 4 operational benchmark produced a one-hour MAE of 171.87 because it uses a deterministic sample of forecast origins and assumes that the last observed weather continues into the future.
- **Weather for the predicted hour is assumed.** The model uses the weather of the target hour. The script carries forward the last observed weather, and the user can override it with -- temp-k, --rain-1h and --clouds-all.
- **Missing hours are handled.** Gaps in the fetched window are filled and counted in the output. If the window has too little history for the 168-hour lag (for example just after the 2014 to 2015 outage), the script stops with a clear message instead of guessing.
- **Clear failures.** If the API is not running, the script prints how to start it. Invalid options such as a horizon above 48 are rejected.
- **Tested.** 12 offline tests cover feature parity with Task 1, gap filling, sensor-fault removal, averaging of multiple weather readings, recursive forecasting, weather overrides and error messages.

The script was also run end to end against the real API with the MySQL dump loaded into a local MySQL 8 server, and with the MongoDB export loaded into an in-memory MongoDB emulator. Both sources returned the same forecast for the same hour.

6. Conclusion

The project delivers a working path from raw Kaggle data to a live forecast. Traffic on I-94 is dominated by a strong daily commuting cycle and a weekday and weekend difference, while weather has almost no direct linear effect at the hourly level. History is by far the most useful signal: adding lags and a moving average reduced the error by 65% over a weather-and-calendar baseline. The same data is available through both a relational and a document database, an API serves it consistently, and the forecasting script reads from that API using exactly the preprocessing the model was trained with.

7. Limitations and next steps

Limitation	Why it matters	Suggested next step
The 2014 to 2015 outage (307 days) was forward-filled with a constant, about 26% of training rows are imputed	The model partly learns from flat artificial traffic, which can overstate how useful lag 1 is. The 2018 test set is 99.7% real, so the reported metrics hold	Train only on contiguous real stretches, or drop the outage, and compare against the current model
The model was trained and evaluated using data from only one traffic-monitoring location.	Traffic patterns on westbound I-94 may differ from those on other roads, cities or travel directions. Therefore, the reported accuracy may not generalize to other locations.	Evaluate and retrain the pipeline using traffic data from multiple sensors, roads and geographical locations.
The forecasting pipeline produces point predictions without uncertainty intervals.	A single predicted value does not show how uncertain the forecast is, especially during unusual weather, holidays or sudden changes in traffic.	Add prediction intervals using methods such as conformal prediction or quantile regression.
Future weather is unknown at forecast time	The model uses same-hour weather, so the script assumes the last observed values	Use a weather forecast feed, or retrain without same-hour weather (its importance is below 1%)
Only two model configurations were compared, chosen by hand	A wider search might find a better setting	Use a time-series cross-validated search (for example TimeSeriesSplit) and compare against LSTM or gradient boosting
Forecast error grows with the horizon	Recursive forecasts compound their own errors	Train direct models per horizon, or add a multi-step approach

Reproducing the results

1. Clone the repository and install requirements.txt (scikit-learn is pinned to 1.6.1, the version used to save the model).

2. Restore MySQL from database/traffic_db_mysql.sql and MongoDB from database/traffic_db_mongodb.jsonl (see the README).
3. Copy .env.example to .env and set the database credentials.
4. Start the API with uvicorn api.main:app --reload.
5. Run python task4_prediction/predict_traffic.py for a forecast, or pytest task4_prediction -v for the offline tests.
6. Run python task4_prediction/benchmark_forecasts.py to reproduce the multi-horizon evaluation and generate its CSV and PNG evidence.
7. Run pytest -q to execute the complete automated test suite.